PyKAN roofline analysis on Google Colab

This notebook measures a GPU-specific roofline and places PyKAN inference workloads on it.

It measures:

- sustained FP32 matrix-multiplication throughput,
- sustained device-memory copy bandwidth,
- PyKAN forward-pass latency,
- an operation count for the PyTorch operations executed by PyKAN,
- an arithmetic-intensity interval based on two traffic models.

The interval is necessary because ordinary Colab notebooks usually cannot access reliable GPU DRAM hardware counters. The left endpoint counts logical tensor traffic across PyTorch operations. The right endpoint counts only compulsory input, output, and parameter traffic. Actual DRAM intensity normally lies between them.

Interpretation:

- interval entirely left of the ridge: **memory-bound prediction**;
- interval entirely right of the ridge: **compute-bound prediction**;
- interval crosses the ridge: **inconclusive without hardware counters**.

Run every cell after selecting **Runtime → Change runtime type → GPU**.

```
# Install PyKAN. The import package is named `kan`.
!pip -q install pykan pandas matplotlib
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 78.1/78.1 kB 4.9 MB/s eta 0:00:00
```

```
Start coding or generate with AI.
```

```
GPU: Tesla T4
VRAM: 14.56 GiB
PyTorch: 2.11.0+cu130
CUDA: 13.0
```

```
import math
import time
from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

from kan import KAN
from torch.utils._python_dispatch import TorchDispatchMode

assert torch.cuda.is_available(), "Enable a GPU runtime in Colab first."

device = torch.device("cuda")
torch.manual_seed(0)
torch.cuda.manual_seed_all(0)
torch.set_grad_enabled(False)

props = torch.cuda.get_device_properties(device)
print("GPU:", props.name)
print(f"VRAM: {props.total_memory / 2**30:.2f} GiB")
print("PyTorch:", torch.__version__)
print("CUDA:", torch.version.cuda)
```

```
GPU: Tesla T4
VRAM: 14.56 GiB
PyTorch: 2.11.0+cu130
CUDA: 13.0
```

1. Measure this runtime's roofs

Vendor specification peaks are easy to misuse because GPU boost clocks and tensor-core modes vary. These microbenchmarks produce sustained roofs in the current Colab session.

The compute benchmark uses ordinary FP32 matrix multiplication with TF32 disabled. The bandwidth benchmark copies one large FP32 tensor to another and counts both the read and write bytes.

```
def median_cuda_time_ms(fn, warmup=10, repeats=30):
    for _ in range(warmup):
        fn()
    torch.cuda.synchronize()

    samples = []
    for _ in range(repeats):
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)
        start.record()
        fn()
        end.record()
        end.synchronize()
        samples.append(start.elapsed_time(end))
    return float(np.median(samples))


def measure_fp32_compute_roof():
    # Start large, then reduce if the assigned GPU has limited memory.
    n = 8192 if props.total_memory >= 12 * 2**30 else 6144
    while True:
        try:
            a = torch.randn((n, n), device=device, dtype=torch.float32)
            b = torch.randn((n, n), device=device, dtype=torch.float32)
            c = torch.empty((n, n), device=device, dtype=torch.float32)
            break
        except torch.cuda.OutOfMemoryError:
            torch.cuda.empty_cache()
            n //= 2
            if n < 2048:
                raise

    old_tf32 = torch.backends.cuda.matmul.allow_tf32
    torch.backends.cuda.matmul.allow_tf32 = False
    ms = median_cuda_time_ms(lambda: torch.mm(a, b, out=c), warmup=5, repeats=15)
    torch.backends.cuda.matmul.allow_tf32 = old_tf32

    flops = 2.0 * n**3
    tflops = flops / (ms * 1e-3) / 1e12
    del a, b, c
    torch.cuda.empty_cache()
    return tflops, n, ms


def measure_memory_roof():
    # Cap allocation so this works across common Colab GPUs.
    nbytes_each = min(int(props.total_memory * 0.20), 768 * 2**20)
    elements = nbytes_each // 4
    src = torch.empty(elements, device=device, dtype=torch.float32)
    dst = torch.empty_like(src)
    ms = median_cuda_time_ms(lambda: dst.copy_(src), warmup=10, repeats=40)
    # A device copy reads src and writes dst.
    moved_bytes = 2.0 * elements * 4
    gbps = moved_bytes / (ms * 1e-3) / 1e9
    del src, dst
    torch.cuda.empty_cache()
    return gbps, moved_bytes, ms


compute_roof_tflops, matmul_n, matmul_ms = measure_fp32_compute_roof()
memory_roof_gbps, copy_bytes, copy_ms = measure_memory_roof()
ridge_flop_per_byte = compute_roof_tflops * 1000 / memory_roof_gbps

print(f"Sustained FP32 compute roof: {compute_roof_tflops:.3f} TFLOP/s")
print(f"Sustained copy bandwidth:    {memory_roof_gbps:.1f} GB/s")
print(f"Ridge point:                 {ridge_flop_per_byte:.2f} FLOP/byte")
print(f"Compute benchmark: {matmul_n} x {matmul_n}, {matmul_ms:.3f} ms")
print(f"Copy benchmark: {copy_bytes / 2**30:.2f} GiB traffic, {copy_ms:.3f} ms")
```

```
Sustained FP32 compute roof: 3.858 TFLOP/s
Sustained copy bandwidth:    232.8 GB/s
Ridge point:                 16.57 FLOP/byte
Compute benchmark: 8192 x 8192, 284.982 ms
Copy benchmark: 1.50 GiB traffic, 6.918 ms
```

2. Count PyKAN work and logical tensor traffic

The dispatcher below observes the low-level PyTorch operations used by one forward pass.

Operation-count conventions:

- add, subtract, multiply, divide, and comparison: one scalar operation per result element;
- exponential, sigmoid, SiLU, and similar nonlinear operations: one nominal operation per result element;
- matrix multiplication: the conventional two operations per multiply-accumulate;
- reduction: approximately one operation per input element.

This is a transparent operation model, not a claim that `exp`, division, and addition have equal hardware cost. That distinction is one reason a KAN may sit below a pure FP32 roof.

```
def tensor_bytes(obj):
    if isinstance(obj, torch.Tensor):
        return obj.numel() * obj.element_size()
    if isinstance(obj, (tuple, list)):
        return sum(tensor_bytes(x) for x in obj)
    if isinstance(obj, dict):
        return sum(tensor_bytes(x) for x in obj.values())
    return 0


def output_numel(obj):
    if isinstance(obj, torch.Tensor):
        return obj.numel()
    if isinstance(obj, (tuple, list)):
        return sum(output_numel(x) for x in obj)
    return 0


class KANOperationCounter(TorchDispatchMode):
    # Counts nominal scalar operations and logical tensor traffic.

    def __init__(self):
        super().__init__()
        self.ops = 0
        self.logical_bytes = 0
        self.calls = {}

    def __torch_dispatch__(self, func, types, args=(), kwargs=None):
        kwargs = kwargs or {}
        out = func(*args, **kwargs)
        name = str(func)
        self.calls[name] = self.calls.get(name, 0) + 1

        # Views only change tensor metadata and do not themselves move tensor data.
        view_only = ("view.", "reshape.", "expand.", "unsqueeze.", "squeeze.",
                     "permute.", "transpose.", "detach.", "alias.")
        if not any(key in name for key in view_only):
            self.logical_bytes += tensor_bytes(args) + tensor_bytes(kwargs) + tensor_bytes(out)

        nout = output_numel(out)
        tensors = [x for x in args if isinstance(x, torch.Tensor)]

        if any(key in name for key in ("addmm",)) and len(tensors) >= 3:
            # input + matmul, output shape MxN and K from second tensor.
            m, n = out.shape[-2:]
            k = tensors[1].shape[-1]
            self.ops += 2 * m * n * k + m * n
        elif any(key in name for key in ("mm.", "bmm.", "matmul.")) and len(tensors) >= 2:
            a, b = tensors[0], tensors[1]
            m, k, n = a.shape[-2], a.shape[-1], b.shape[-1]
            batch = math.prod(torch.broadcast_shapes(a.shape[:-2], b.shape[:-2])) or 1
            self.ops += 2 * batch * m * n * k
        elif any(key in name for key in ("sum.", "mean.")) and tensors:
            self.ops += tensors[0].numel()
        elif any(key in name for key in (
            "add.", "sub.", "mul.", "div.", "pow.", "exp.", "sigmoid.",
            "silu.", "neg.", "gt.", "ge.", "lt.", "le.", "eq."
        )):
            self.ops += nout

        return out
```

3. Define the PyKAN sweep

Change `CONFIGS` to match your application. `width` is the PyKAN layer-width list. `grid` is the number of spline intervals and `k` is the spline order.

`model.speed()` disables the symbolic branch. Leaving the symbolic branch active would measure functionality that is normally unnecessary for numerical inference.

```
CONFIGS = [
    # name, width, grid, spline order, batch size
    {"name": "tiny_B1",    "width": [8, 8, 1],     "grid": 5,  "k": 3, "batch": 1},
    {"name": "tiny_B256",  "width": [8, 8, 1],     "grid": 5,  "k": 3, "batch": 256},
    {"name": "small_B64",  "width": [32, 32, 1],   "grid": 5,  "k": 3, "batch": 64},
    {"name": "small_B512", "width": [32, 32, 1],   "grid": 10, "k": 3, "batch": 512},
    {"name": "wide_B256",  "width": [128, 128, 1], "grid": 5,  "k": 3, "batch": 256},
]


def build_kan(cfg):
    # PyKAN may normalize/mutate the supplied width list in place. Pass a copy
    # so CONFIGS continues to contain plain integer dimensions.
    width_for_pykan = list(cfg["width"])
    model = KAN(
        width=width_for_pykan,
        grid=cfg["grid"],
        k=cfg["k"],
        device="cuda",
        auto_save=False,
    )
    model.speed()
    # Do not call model.eval(): some PyKAN releases override nn.Module.train()
    # with a dataset-training method, while eval() internally calls train(False).
    return model


def benchmark_kan(cfg):
    # Read this before constructing the model because PyKAN may mutate width.
    input_dim = int(cfg["width"][0])
    model = build_kan(cfg)
    x = torch.randn(cfg["batch"], input_dim, device=device)

    # Count one representative forward pass. Do not time instrumentation.
    counter = KANOperationCounter()
    with counter:
        y = model(x)

    # Some PyKAN releases return a tensor; tolerate tuple/list returns as well.
    primary_y = y[0] if isinstance(y, (tuple, list)) else y
    parameter_bytes = sum(p.numel() * p.element_size() for p in model.parameters())
    compulsory_bytes = tensor_bytes(x) + tensor_bytes(primary_y) + parameter_bytes

    # Select repeats so tiny models do enough work to get stable timing.
    repeats = 300 if cfg["batch"] <= 64 else 100
    latency_ms = median_cuda_time_ms(lambda: model(x), warmup=20, repeats=repeats)

    ops = float(counter.ops)
    seconds = latency_ms * 1e-3
    achieved_gops = ops / seconds / 1e9

    # Actual DRAM traffic is normally between these two software-visible bounds.
    ai_conservative = ops / max(counter.logical_bytes, 1)
    ai_optimistic = ops / max(compulsory_bytes, 1)
    lo, hi = sorted((ai_conservative, ai_optimistic))

    if hi < ridge_flop_per_byte:
        diagnosis = "memory-bound"
    elif lo > ridge_flop_per_byte:
        diagnosis = "compute-bound"
    else:
        diagnosis = "inconclusive"

    row = {
        **cfg,
        "parameters": sum(p.numel() for p in model.parameters()),
        "latency_ms": latency_ms,
        "nominal_ops": int(ops),
        "logical_traffic_MB": counter.logical_bytes / 1e6,
        "compulsory_traffic_MB": compulsory_bytes / 1e6,
        "AI_low": lo,
        "AI_high": hi,
```

```
        "achieved_Gop_s": achieved_gops,
        "ridge_FLOP_byte": ridge_flop_per_byte,
        "diagnosis": diagnosis,
    }

    del model, x, y, primary_y
    torch.cuda.empty_cache()
    return row


rows = []
for cfg in CONFIGS:
    print("Benchmarking", cfg["name"])
    rows.append(benchmark_kan(cfg))

results = pd.DataFrame(rows)
display(results[[
    "name", "width", "grid", "k", "batch", "parameters", "latency_ms",
    "AI_low", "AI_high", "achieved_Gop_s", "diagnosis"
]])
```

```
Benchmarking tiny_B1
Benchmarking tiny_B256
Benchmarking small_B64
Benchmarking small_B512
Benchmarking wide_B256
```

	name	width	grid	k	batch	parameters	latency_ms	AI_low	AI_high	achieved_Gop_s	diagnosis
0	tiny_B1	[8, 8, 1]	5	3	1	1380	2.755728	0.063024	1.077934	0.002173	memory-bound
1	tiny_B256	[8, 8, 1]	5	3	256	1380	3.709712	0.123784	89.092291	0.353899	inconclusive
2	small_B64	[32, 32, 1]	5	3	64	17796	2.324528	0.173266	29.469158	1.009533	inconclusive
3	small_B512	[32, 32, 1]	10	3	512	23396	2.419312	0.191259	174.290877	11.610785	inconclusive
4	wide_B256	[128, 128, 1]	5	3	256	267780	2.439168	0.265289	86.070983	42.457914	inconclusive

4. Plot the roofline

Each horizontal marker is the workload's possible arithmetic-intensity interval. The dot is its geometric midpoint, used only for visualization. The workload's measured operation rate is identical across its interval because changing the assumed traffic does not change the measured time or counted work.

Points far below the roof can indicate Python or kernel-launch overhead, insufficient parallelism, costly nonlinear instructions, poor fusion, or a mismatch between nominal operation counting and actual instruction cost.

```
def plot_roofline(results):
    positive_lows = results.loc[results.AI_low > 0, "AI_low"]
    xmin = max(1e-4, positive_lows.min() / 5 if len(positive_lows) else ridge_flop_per_byte / 100)
    xmax = max(results.AI_high.max() * 5, ridge_flop_per_byte * 100)
    xs = np.logspace(np.log10(xmin), np.log10(xmax), 600)

    compute_gops = compute_roof_tflops * 1000
    roof = np.minimum(compute_gops, memory_roof_gbps * xs)

    fig, ax = plt.subplots(figsize=(11, 7))
    ax.loglog(xs, roof, color="#1565C0", linewidth=3,
              label=f"Measured FP32 roof: {compute_roof_tflops:.2f} TFLOP/s")
    ax.axvline(ridge_flop_per_byte, color="#616161", linestyle="--", linewidth=1.5,
               label=f"Ridge: {ridge_flop_per_byte:.2f} FLOP/byte")

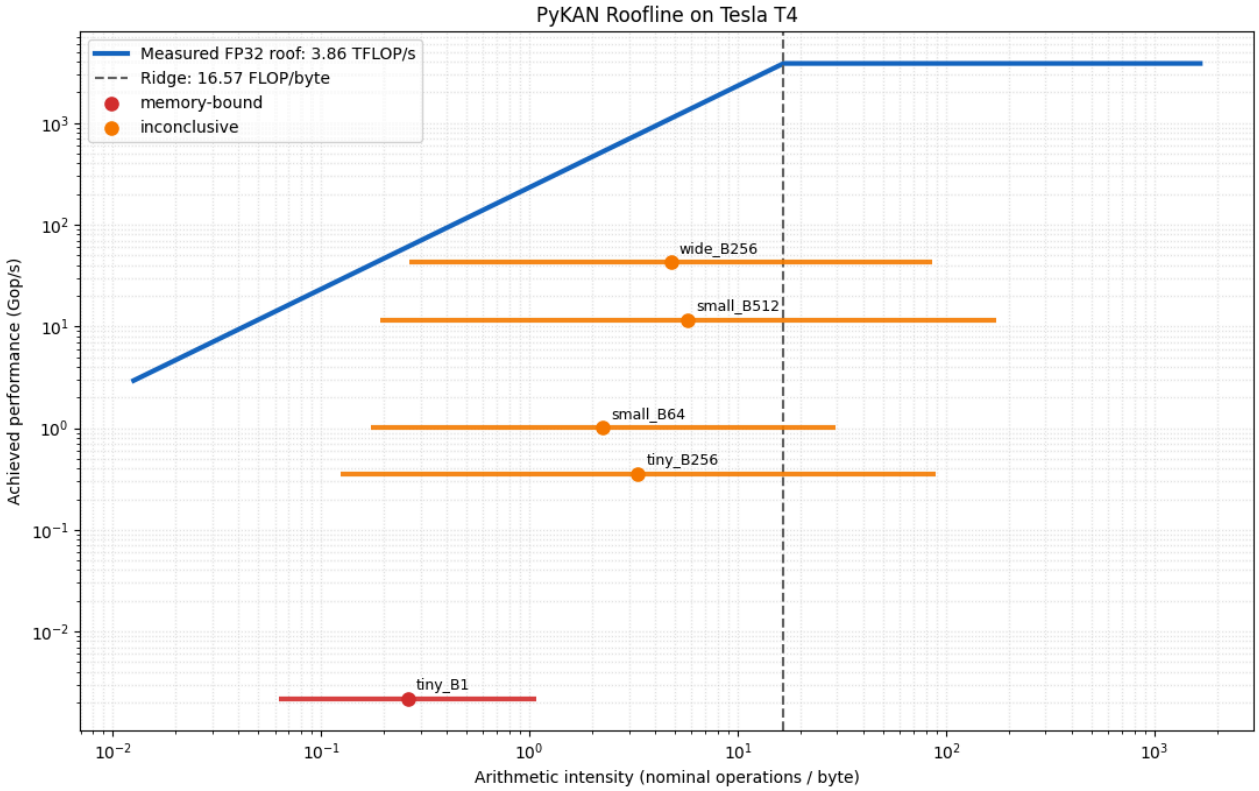
    colors = {"memory-bound": "#D32F2F", "compute-bound": "#388E3C", "inconclusive": "#F57C00"}
    used_labels = set()

    for _, r in results.iterrows():
        lo = max(r.AI_low, xmin)
        hi = max(r.AI_high, lo)
        mid = math.sqrt(lo * hi)
        color = colors[r.diagnosis]
        label = r.diagnosis if r.diagnosis not in used_labels else None
        used_labels.add(r.diagnosis)
        ax.hlines(r.achieved_Gop_s, lo, hi, color=color, linewidth=3, alpha=0.9)
        ax.scatter(mid, r.achieved_Gop_s, s=65, color=color, label=label, zorder=3)
        ax.annotate(r["name"], (mid, r.achieved_Gop_s), xytext=(5, 6),
                    textcoords="offset points", fontsize=9)

    ax.set_xlabel("Arithmetic intensity (nominal operations / byte)")
    ax.set_ylabel("Achieved performance (Gop/s)")
```

```
    ax.set_title(f"PyKAN Roofline on {props.name}")
    ax.grid(True, which="both", linestyle=":", alpha=0.35)
    ax.legend(loc="best")
    fig.tight_layout()
    return fig


fig = plot_roofline(results)
plt.show()
```



5. Save the measurements and graph

The CSV records all assumptions and results. Download both outputs before the Colab runtime is deleted.

```
results.to_csv("pykan_roofline_results.csv", index=False)
fig.savefig("pykan_roofline.png", dpi=220, bbox_inches="tight")

from google.colab import files
files.download("pykan_roofline_results.csv")
files.download("pykan_roofline.png")
```

How to report the conclusion

Use wording that matches the interval result:

- **Memory-bound:** "The estimated intensity interval remains below the measured ridge point, so the PyKAN configuration is predicted to be memory-bound on this GPU."
- **Compute-bound:** "The estimated intensity interval remains above the measured ridge point, so the PyKAN configuration is predicted to be compute-bound under the roofline model."
- **Inconclusive:** "The estimated intensity interval crosses the ridge point. Software-visible traffic estimates cannot classify the kernel reliably; GPU DRAM hardware counters are required."

Do not infer the bottleneck merely because a point is below the roof. Its horizontal position relative to the ridge determines the roofline classification. Vertical distance from the roof measures utilization and may expose overhead or other ceilings.

For publication-quality results, repeat the experiment with Nsight Compute and replace the traffic interval with `dram__bytes_read.sum + dram__bytes_write.sum`. Also report the exact Colab GPU, PyTorch version, PyKAN version, precision, batch size, layer widths, grid size, spline order, and whether `model.speed()` was enabled.

```
!which ncu
!ncu --version
```

```
/usr/local/cuda/bin/ncu
NVIDIA (R) Nsight Compute Command Line Profiler
Copyright (c) 2018-2025 NVIDIA Corporation
Version 2025.3.1.0 (build 36398880) (public-release)
```

```
%%writefile profile_pykan.py

import torch
from kan import KAN

assert torch.cuda.is_available()

device = "cuda"

# Change these to the configuration you want to profile.
WIDTH = [32, 32, 1]
GRID = 5
K = 3
BATCH_SIZE = 256

# PyKAN can mutate the width list, so pass it a copy.
model = KAN(
    width=list(WIDTH),
    grid=GRID,
    k=K,
    device=device,
    auto_save=False,
)

model.speed()

x = torch.randn(
    BATCH_SIZE,
    WIDTH[0],
    device=device,
    dtype=torch.float32,
)

# Initialize CUDA and warm up PyKAN outside the measured NVTX range.
with torch.inference_mode():
    for _ in range(20):
        y = model(x)

torch.cuda.synchronize()

# Nsight Compute will profile only this named range.
torch.cuda.nvtx.range_push("pykan_forward")

with torch.inference_mode():
    y = model(x)

torch.cuda.synchronize()
torch.cuda.nvtx.range_pop()

print("Output shape:", y.shape)
```

```
Writing profile_pykan.py
```

```
!python profile_pykan.py
```

```
Output shape: torch.Size([256, 1])
```

```
!/usr/local/cuda/bin/ncu \
  --target-processes all \
  --metrics gpu__time_duration.sum \
  python profile_pykan.py
```

```
    gpu__time_duration.sum          us         5.47
    ----------------------- ----------- ------------

  void at::reduce_kernel<512, 1, at::ReduceOp<float, at::func_wrapper_t<float, at::sum_functor<float, float, float>::opera
    Section: Command line profiler metrics
    ----------------------- ----------- ------------
    Metric Name             Metric Unit Metric Value
    ----------------------- ----------- ------------
    gpu__time_duration.sum          us        10.56
    ----------------------- ----------- ------------

  void at::vectorized_elementwise_kernel<4, at::CUDAFunctorOnSelf_add<float>, std::array<char *, 2>>(int, T2, T3) (1, 1, 1
    Section: Command line profiler metrics
    ----------------------- ----------- ------------
    Metric Name             Metric Unit Metric Value
    ----------------------- ----------- ------------
    gpu__time_duration.sum          us         3.90
    ----------------------- ----------- ------------

  void at::elementwise_kernel<128, 2, void at::gpu_kernel_impl_nocast<at::BinaryFunctor<float, float, float, binary_intern
    Section: Command line profiler metrics
    ----------------------- ----------- ------------
    Metric Name             Metric Unit Metric Value
    ----------------------- ----------- ------------
    gpu__time_duration.sum          us         5.12
    ----------------------- ----------- ------------

  void at::elementwise_kernel<128, 2, void at::gpu_kernel_impl_nocast<at::CUDAFunctor_add<float>>(at::TensorIteratorBase &
    Section: Command line profiler metrics
    ----------------------- ----------- ------------
    Metric Name             Metric Unit Metric Value
    ----------------------- ----------- ------------
    gpu__time_duration.sum          us         5.12
    ----------------------- ----------- ------------

  void at::elementwise_kernel<128, 2, void at::gpu_kernel_impl_nocast<at::BinaryFunctor<float, float, float, binary_intern
    Section: Command line profiler metrics
    ----------------------- ----------- ------------
    Metric Name             Metric Unit Metric Value
    ----------------------- ----------- ------------
    gpu__time_duration.sum          us         5.12
    ----------------------- ----------- ------------

  void at::elementwise_kernel<128, 2, void at::gpu_kernel_impl_nocast<at::CUDAFunctor_add<float>>(at::TensorIteratorBase &
    Section: Command line profiler metrics
    ----------------------- ----------- ------------
    Metric Name             Metric Unit Metric Value
    ----------------------- ----------- ------------
    gpu__time_duration.sum          us         5.12
    ----------------------- ----------- ------------
```

```
!/usr/local/cuda/bin/ncu \
  --target-processes all \
  --metrics gpu__time_duration.sum \
  python profile_pykan.py 2>&1 | grep -E "ERR_|ERROR|Output shape"
```

```
Output shape: torch.Size([256, 1])
```

```
!/usr/local/cuda/bin/ncu \
  --target-processes all \
  --nvtx \
  --nvtx-include "pykan_forward/" \
  --section SpeedOfLight \
  --section SpeedOfLight_RooflineChart \
  --section MemoryWorkloadAnalysis \
  --section ComputeWorkloadAnalysis \
  --section LaunchStats \
  --force-overwrite \
  --export pykan_ncu \
  python profile_pykan.py
```

```
==PROF== Connected to process 8910 (/usr/bin/python3.13)
==PROF== Profiling "index_elementwise_kernel" - 0: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 1: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 2: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 3: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 4: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 5: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 6: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 7: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 8: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 9: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 10: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 11: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 12: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 13: 0%....50%....100% - 10 passes
```

```
==PROF== Profiling "vectorized_elementwise_kernel" - 14: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 15: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 16: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 17: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 18: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 19: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 20: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 21: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 22: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 23: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 24: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 25: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 26: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 27: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 28: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 29: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 30: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 31: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 32: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 33: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 34: 0%....50%....100% - 10 passes
==PROF== Profiling "volta_sgemm_64x64_tn" - 35: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 36: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 37: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 38: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 39: 0%....50%....100% - 10 passes
==PROF== Profiling "reduce_kernel" - 40: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 41: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 42: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 43: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 44: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 45: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 46: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 47: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 48: 0%....50%....100% - 10 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 49: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 50: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 51: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 52: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 53: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 54: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 55: 0%....50%....100% - 10 passes
==PROF== Profiling "elementwise_kernel" - 56: 0%....50%....100% - 10 passes
```

```
!ls -lh pykan_ncu.ncu-rep
```

```
-rw-r--r-- 1 root root 52M Oct 10 18:29 pykan_ncu.ncu-rep
```

```
from google.colab import files
files.download("pykan_ncu.ncu-rep")
```

```
!/usr/local/cuda/bin/ncu \
  --import pykan_ncu.ncu-rep \
  --page raw \
  --csv \
  > pykan_ncu_raw.csv
```

```
from google.colab import files

files.download("pykan_ncu_raw.csv")
```

Here is where I realized that launch was dominating computation or memory. So prepared an ablation script.

```
from google.colab import files
files.upload()
```

Choose Files pykan_ablation.py

pykan_ablation.py(text/x-python) - 4031 bytes, last modified: 10/11/2026 - 100% done

Saving pykan_ablation.py to pykan_ablation.py

{'pykan_ablation.py': b'#!/usr/bin/env python3\n"""Parameterized PyKAN inference benchmark for timing and Nsight Compute."""\n\nimport argparse\nimport json\nimport statistics\nfrom pathlib import Path\n\nimport torch\nfrom kan import KAN\n\n\ndef parse_width(text: str) -> list[int]:\n   try:\n   width = [int(x) for x in text.split(",")]\n   except ValueError as exc:\n   raise argparse.ArgumentTypeError("width must look like 32,32,1") from exc\n   if len(width) < 2 or any(x < 1 for x in width):\n   raise argparse.ArgumentTypeError("width needs at least two positive integers")\n   return width\n\n\ndef arguments():\n   p = argparse.ArgumentParser()\n   p.add_argument("--name", default="baseline")\n   p.add_argument("--width", type=parse_width, default=parse_width("32,32,1"))\n   p.add_argument("--grid", type=int, default=5)\n   p.add_argument("--k", type=int, default=3)\n   p.add_argument("--batch", type=int, default=256)\n   p.add_argument("--warmup", type=int, default=20)\n   p.add_argument("--repeats", type=int, default=100)\n   p.add_argument("--seed", type=int, default=0)\n   p.add_argument("--output", type=Path, help="append one JSON record to this JSONL file")\n   p.add_argument(\n   "--profile-only",\n   action="store_true",\n   help="run warmup plus one NVTX-marked forward pass for Nsight Compute",\n   )\n   return p.parse_args()\n\n\ndef timed_forward(model, x) -> float:\n   start = torch.cuda.Event(enable_timing=True)\n   end = torch.cuda.Event(enable_timing=True)\n   start.record()\n   with torch.inference_mode():\n   model(x)\n   end.record()\n   end.synchronize()\n   return start.elapsed_time(end)\n\n\ndef main():\n   args = arguments()\n   if not torch.cuda.is_available():\n   raise RuntimeError("A CUDA GPU is required")\n   if args.grid < 1 or args.k < 1 or args.batch < 1:\n   raise ValueError("grid, k, and batch must be positive")\n\n   torch.manual_seed(args.seed)\n   torch.cuda.manual_seed_all(args.seed)\n   device = torch.device("cuda")\n\n   # PyKAN may mutate its width argument, so it must receive a private copy.\n   original_width = list(args.width)\n   model = KAN(\n   width=list(original_width),\n   grid=args.grid,\n   k=args.k,\n   device="cuda",\n   auto_save=False,\n   seed=args.seed,\n   )\n   model.speed()\n   x = torch.randn(args.batch, original_width[0], device=device)\n\n   with torch.inference_mode():\n   for _ in range(args.warmup):\n   y = model(x)\n   torch.cuda.synchronize()\n\n   if args.profile_only:\n   # All configurations use the same range name, allowing one ncu command.\n   torch.cuda.nvtx.range_push("pykan_forward")\n   with torch.inference_mode():\n   y = model(x)\n   torch.cuda.synchronize()\n   torch.cuda.nvtx.range_pop()\n   print(json.dumps({\n   "name": args.name,\n   "width": original_width,\n   "grid": args.grid,\n   "k": args.k,\n   "batch": args.batch,\n   "output_shape": list(y.shape),\n   }))\n   return\n\n   samples_ms = [timed_forward(model, x) for _ in range(args.repeats)]\n   samples_ms.sort()\n   median_ms = statistics.median(samples_ms)\n   p10_ms = samples_ms[max(0, int(0.10 * (len(samples_ms) - 1)))]\n   p90_ms = samples_ms[min(len(samples_ms) - 1, int(0.90 * (len(samples_ms) - 1)))]\n\n   result = {\n   "name": args.name,\n   "gpu": torch.cuda.get_device_name(),\n   "torch":

```
%%bash

rm -f ablation_results.jsonl

# Baseline
python pykan_ablation.py \
  --name baseline \
  --output ablation_results.jsonl

# Batch-size ablation
for batch in 1 32 256 1024 4096
do
  python pykan_ablation.py \
    --name batch_${batch} \
    --batch ${batch} \
    --output ablation_results.jsonl
done

# Grid-size ablation
for grid in 3 5 10 20
do
  python pykan_ablation.py \
    --name grid_${grid} \
    --grid ${grid} \
    --output ablation_results.jsonl
done

# Spline-order ablation
for k in 1 2 3 4
do
  python pykan_ablation.py \
    --name k_${k} \
    --k ${k} \
    --output ablation_results.jsonl
done

# Width ablation
python pykan_ablation.py \
  --name width_8 \
  --width 8,8,1 \
  --output ablation_results.jsonl

python pykan_ablation.py \
  --name width_64 \
  --width 64,64,1 \
  --output ablation_results.jsonl

python pykan_ablation.py \
  --name width_128 \
```

```
  --width 128,128,1 \
  --output ablation_results.jsonl
```


```
{"name": "baseline", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 256, "par
{"name": "batch_1", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 1, "parame
{"name": "batch_32", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 32, "para
{"name": "batch_256", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 256, "pa
{"name": "batch_1024", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 1024, "
{"name": "batch_4096", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 4096, "
{"name": "grid_3", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 3, "k": 3, "batch": 256, "param
{"name": "grid_5", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 256, "param
{"name": "grid_10", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 10, "k": 3, "batch": 256, "par
{"name": "grid_20", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 20, "k": 3, "batch": 256, "par
{"name": "k_1", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 1, "batch": 256, "paramete
{"name": "k_2", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 2, "batch": 256, "paramete
{"name": "k_3", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 256, "paramete
{"name": "k_4", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [32, 32, 1], "grid": 5, "k": 4, "batch": 256, "paramete
{"name": "width_8", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [8, 8, 1], "grid": 5, "k": 3, "batch": 256, "parame
{"name": "width_64", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [64, 64, 1], "grid": 5, "k": 3, "batch": 256, "par
{"name": "width_128", "gpu": "Tesla T4", "torch": "2.11.0+cu130", "width": [128, 128, 1], "grid": 5, "k": 3, "batch": 256, "
```


```
import json
import pandas as pd

with open("ablation_results.jsonl") as f:
    records = [json.loads(line) for line in f if line.strip()]

results = pd.DataFrame(records)

display(results[[
    "name",
    "width",
    "grid",
    "k",
    "batch",
    "parameters",
    "median_ms",
    "p10_ms",
    "p90_ms",
    "samples_per_second",
]])

results.to_csv("ablation_results.csv", index=False)
```

	name	width	grid	k	batch	parameters	median_ms	p10_ms	p90_ms	samples_per_second
0	baseline	[32, 32, 1]	5	3	256	17796	1.992416	1.911360	2.161344	1.284872e+05
1	batch_1	[32, 32, 1]	5	3	1	17796	2.101248	1.855520	3.326048	4.759076e+02
2	batch_32	[32, 32, 1]	5	3	32	17796	1.954576	1.863712	2.109312	1.637184e+04
3	batch_256	[32, 32, 1]	5	3	256	17796	1.917872	1.857568	2.100032	1.334813e+05
4	batch_1024	[32, 32, 1]	5	3	1024	17796	1.993776	1.921568	2.179040	5.135983e+05
5	batch_4096	[32, 32, 1]	5	3	4096	17796	3.703824	3.668960	4.113952	1.105884e+06
6	grid_3	[32, 32, 1]	3	3	256	15556	2.099184	1.947616	2.347488	1.219521e+05
7	grid_5	[32, 32, 1]	5	3	256	17796	1.941360	1.896416	2.101792	1.318663e+05
8	grid_10	[32, 32, 1]	10	3	256	23396	1.995136	1.910016	2.953728	1.283121e+05
9	grid_20	[32, 32, 1]	20	3	256	34596	1.922160	1.897152	2.129888	1.331835e+05
10	k_1	[32, 32, 1]	5	1	256	15428	1.361680	1.324896	1.487840	1.880031e+05
11	k_2	[32, 32, 1]	5	2	256	16612	2.507648	2.126336	3.106368	1.020877e+05
12	k_3	[32, 32, 1]	5	3	256	17796	1.951552	1.894336	2.342880	1.311776e+05
13	k_4	[32, 32, 1]	5	4	256	18980	2.385152	2.258912	2.585952	1.073307e+05
14	width_8	[8, 8, 1]	5	3	256	1380	1.973632	1.907552	2.087136	1.297101e+05
15	width_64	[64, 64, 1]	5	3	256	68356	1.926144	1.894624	2.065728	1.329080e+05
16	width_128	[128, 128, 1]	5	3	256	267780	2.546208	2.449920	2.624288	1.005417e+05

```
%%bash

profile_config () {
    name="$1"
    shift

    echo "Profiling ${name}"
```

```
    /usr/local/cuda/bin/ncu \
      --target-processes all \
      --nvtx \
      --nvtx-include "pykan_forward/" \
      --set full \
      --force-overwrite \
      --export "ncu_${name}" \
      python pykan_ablation.py \
        --name "${name}" \
        --profile-only \
        "$@"
}

profile_config batch_4096 --batch 4096
profile_config width_128  --width 128,128,1
profile_config grid_20    --grid 20
profile_config k_1        --k 1
profile_config k_4        --k 4
```

```
Process is terminated.
```

```
!echo "Starting batch_4096"

!/usr/local/cuda/bin/ncu \
  --target-processes all \
  --nvtx \
  --nvtx-include "pykan_forward/" \
  --metrics \
gpu__time_duration.sum,sm__throughput.avg.pct_of_peak_sustained_elapsed,gpu__dram_throughput.avg.pct_of_peak_sustained_ela
  --force-overwrite \
  --export ncu_batch4096 \
  python pykan_ablation.py \
    --name batch_4096 \
    --batch 4096 \
    --profile-only

!echo "Finished batch_4096"
```

```
==PROF== Profiling "elementwise_kernel" - 37: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 38: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 39: 0%....50%....100% - 8 passes
==PROF== Profiling "reduce_kernel" - 40: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 41: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 42: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 43: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 44: 0%....50%....100% - 8 passes
```

```
==PROF== Profiling "elementwise_kernel" - 84: 0%....50%....100% - 8 passes
==PROF== Profiling "reduce_kernel" - 85: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 86: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 87: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 88: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 89: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 90: 0%....50%....100% - 8 passes
{"name": "batch_4096", "width": [32, 32, 1], "grid": 5, "k": 3, "batch": 4096, "output_shape": [4096, 1]}
==PROF== Disconnected from process 18222
==PROF== Report: /content/ncu_batch4096.ncu-rep
Finished batch_4096
```

```
!echo "Starting width_128"

!/usr/local/cuda/bin/ncu \
  --target-processes all \
  --nvtx \
  --nvtx-include "pykan_forward/" \
  --metrics \
gpu__time_duration.sum,sm__throughput.avg.pct_of_peak_sustained_elapsed,gpu__dram_throughput.avg.pct_of_peak_sustained_ela
  --force-overwrite \
  --export ncu_width128 \
  python pykan_ablation.py \
    --name width_128 \
    --width 128,128,1 \
    --profile-only

!echo "Finished width_128"
```

```
==PROF== Profiling "elementwise_kernel" - 37: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 38: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 39: 0%....50%....100% - 8 passes
==PROF== Profiling "reduce_kernel" - 40: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 41: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 42: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 43: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 44: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 45: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 46: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 47: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 48: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 49: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 50: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 51: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 52: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 53: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 54: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 55: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 56: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 57: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 58: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 59: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 60: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 61: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 62: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 63: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 64: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 65: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 66: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 67: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 68: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 69: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 70: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 71: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 72: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 73: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 74: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 75: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 76: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 77: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 78: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 79: 0%....50%....100% - 8 passes
==PROF== Profiling "kernel" - 80: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 81: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 82: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 83: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 84: 0%....50%....100% - 8 passes
==PROF== Profiling "reduce_kernel" - 85: 0%....50%....100% - 8 passes
==PROF== Profiling "vectorized_elementwise_kernel" - 86: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 87: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 88: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 89: 0%....50%....100% - 8 passes
==PROF== Profiling "elementwise_kernel" - 90: 0%....50%....100% - 8 passes
{"name": "width_128", "width": [128, 128, 1], "grid": 5, "k": 3, "batch": 256, "output_shape": [256, 1]}
==PROF== Disconnected from process 19710
==PROF== Report: /content/ncu_width128.ncu-rep
Finished width_128
```

```
!/usr/local/cuda/bin/ncu \
  --import ncu_batch4096.ncu-rep \
  --page raw \
  --csv > ncu_batch4096.csv

!/usr/local/cuda/bin/ncu \
  --import ncu_width128.ncu-rep \
  --page raw \
  --csv > ncu_width128.csv!zip pykan_ablation_ncu.zip ncu_*.csv ablation_results.csv
```

```
!zip pykan_selected_ncu.zip \
  ncu_batch4096.csv \
  ncu_width128.csv \
  ablation_results.csv
```

```
       zip warning: name not matched: ncu_width128.csv
  adding: ncu_batch4096.csv (deflated 94%)
```